

Cloud-native Disaggregated Storage over RDMA with DISTORT

Anonymous Author(s)

Institution

City, Country

ABSTRACT

As cloud-native architectures become the standard for deploying scalable applications, the demand for high-performance, low-latency storage has surged. Disaggregated storage, particularly NVMe-over-Fabrics (NVMe-oF) using Remote Direct Memory Access (RDMA), offers a promising solution by decoupling storage capacity from compute nodes, maintaining near-local access speeds. However, integrating physical RDMA targets with the dynamic, containerized environments of Kubernetes presents significant orchestration challenges. In this paper, we present DISTORT, a Kubernetes-native storage engine designed to bridge this gap. DISTORT leverages a Custom Resource Definition (CRD)-driven state machine to manage the lifecycle of physical NVMe devices, partition them asynchronously, and expose them as NVMe-oF RDMA targets directly to Kubernetes Pods via a Container Storage Interface (CSI) driver. By offloading the control plane to Kubernetes while maintaining a lightweight data path, DISTORT achieves high-performance, dynamic storage provisioning suitable for data-intensive cloud applications.

ACM Reference Format:

Anonymous Author(s). 2026. Cloud-native Disaggregated Storage over RDMA with DISTORT. In *Proceedings of ACM Conference (Conference'17)*. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 INTRODUCTION

Cloud-native ecosystems, spearheaded by Kubernetes, have revolutionized application deployment through containerization and dynamic orchestration. Traditionally, these environments relied on software-defined storage (SDS) solutions that often introduce overhead via virtualization or user-space file systems. As applications such as machine learning and high-performance databases migrate to the cloud, the overhead of standard SDS becomes a bottleneck.

NVMe-over-Fabrics (NVMe-oF) utilizing RDMA provides microseconds of latency and high throughput, rivaling local NVMe drives. Disaggregating storage using NVMe-oF allows independent scaling of compute and storage resources, improving resource utilization. Yet, orchestrating these high-performance physical targets within Kubernetes without compromising performance or introducing complex middleware is an open challenge.

We introduce DISTORT, a Cloud-native Disaggregated Storage engine over RDMA. DISTORT integrates physical NVMe management directly into the Kubernetes control plane. It introduces an

architecture comprising a centralized management controller, a node-level hardware agent, and a standard CSI driver. This design allows Kubernetes to natively declare, partition, and consume bare-metal NVMe storage over RDMA fabrics.

2 RELATED WORK

Recent advancements in distributed storage have focused heavily on integrating high-performance interconnects with cloud-orchestrator frameworks. Systems like SPDK (Storage Performance Development Kit) provide user-space NVMe drivers to bypass kernel overhead, while commercial and open-source CSI drivers (e.g., OpenEBS, Rook/Ceph) offer varying degrees of RDMA support. However, these systems often impose heavy control-plane dependencies or require dedicated storage nodes running complex SDS stacks. DISTORT distinguishes itself by utilizing the Linux kernel's native configs and NVMe-oF subsystems, orchestrating them purely through Kubernetes Custom Resources, thereby maintaining a zero-overhead data path.

3 DESIGN

DISTORT's architecture is bifurcated into two logical layers: the *NVMe Management Layer* (the "Hardware Control Plane") and the *CSI Layer* (the "Kubernetes Bridge"). This separation of concerns allows the system to decouple physical hardware management from the Kubernetes volume lifecycle using a state-machine-via-CRD approach.

3.1 Custom Resource Definitions (CRDs)

At the core of DISTORT's declarative model are four Custom Resource Definitions that mirror the physical and logical state of the storage fabric:

- (1) **NVMeDevice**: Represents a discovered physical NVMe storage controller on a worker node, including attributes like serial number, NUMA alignment, and block capacity.
- (2) **NVMeDeviceClaim**: Allows administrators or automated provisioners to reserve specific NVMeDevice instances for dedicated workloads.
- (3) **NVMePartition**: Represents a logical slice of an NVMeDevice. It dictates the required capacity and, once scheduled, tracks the NVMe-oF network endpoint details (NQN, Portal IP, Port) required for client connections.
- (4) **RDMAStorageNode**: Represents a worker node's capability to participate in the storage fabric, providing health status and available network interfaces for RDMA traffic.

3.2 NVMe Management Layer

This layer is responsible for physical device discovery, disk partitioning, and kernel-level NVMe-oF target exports. It comprises two main components:

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, July 2017, Washington, DC, USA

© 2026 Association for Computing Machinery.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM...\$15.00

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

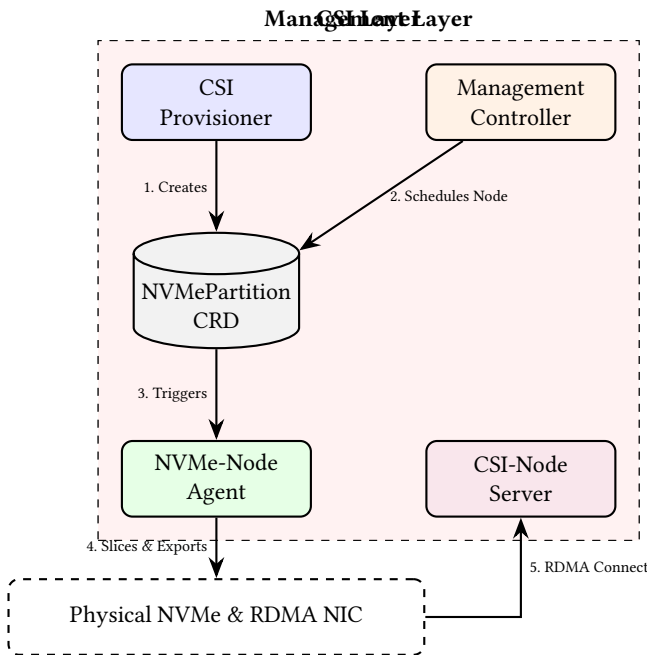


Figure 1: DISTORT Architectural Layers and Interaction

- **NVMe-Management-Controller:** Deployed as a centralized replica, it handles administration and placement logic. It reconciles NVMeDeviceClaim objects, allowing cluster administrators to claim specific disks by serial number. Furthermore, when an NVMePartition is requested without a pre-assigned node, this controller selects the optimal RDMAStorageNode based on available free capacity.
- **NVMe-Node-Agent:** Deployed as a DaemonSet across storage-providing nodes. It performs continuous discovery by scanning the PCIe bus for NVMe controllers and reporting active RDMA-capable NICs. Its execution loop watches for NVMePartition CRDs assigned to its resident node, launching parted to slice the physical media and configuring the kernel to expose the partition as an NVMe-oF target.

3.3 CSI Layer (Kubernetes Bridge)

This layer translates standard PersistentVolumeClaims (PVCs) into concrete storage allocations and subsequently mounts the volumes to application Pods.

- **CSI-Provisioner:** A controller that watches for new PVCs. Instead of communicating with a traditional centralized storage array’s backend API, it creates an NVMePartition CRD specifying the required capacity and access mode. It then blocks until the management layer updates the partition’s status with an RDMA endpoint (NQN and Portal IP), at which point it binds the resulting PersistentVolume (PV).
- **CSI-Node-Server:** A DaemonSet located on the compute nodes consuming the storage. During the volume staging phase, it executes `nvme connect` against the NVMe-oF cluster, connecting the Pod to the remote block device (e.g.,

`/dev/nvme1n1`). It then bind-mounts this device into the corresponding container’s root filesystem.

4 IMPLEMENTATION

DISTORT is entirely implemented in Go (version 1.23+), leveraging the controller-runtime and Kubebuilder frameworks to enforce operator paradigms.

4.1 Codebase Layout & Components

The system is compiled into three distinct binary executables situated under the `cmd/` directory:

- (1) `distort-manager`: The control plane application housing the claims and placement schedulers.
- (2) `distort-agent`: The privileged hardware-interaction daemon.
- (3) `distort-csi`: Intended to be deployed per the Container Storage Interface spec, containing the Identity, Controller, and Node server gRPC implementations.

Building the binaries relies on a standard Makefile utilizing `go build`. Kubebuilder macros extract RBAC definitions and scheme topologies from Go structural comments, ensuring that API generations (`make manifests`) directly reflect the Go codebase.

4.2 Hardware Integrations

To minimize overhead, the `distort-agent` bypasses typical storage middleware.

- **Discovery Mapping:** It probes `/sys/class/nvme` continuously to calculate NUMA topology and total block bytes, translating this into resilient NVMeDevice Custom Resources.
- **Partition Slicing:** Using the `os/exec` package, it implements a wrapper over the parted system utility to inject GUID Partition Tables (GPT) and carve primary partitions on demand dynamically based on Kubernetes limits.
- **NVMe-oF Target Orchestration:** Rather than relying on external Python-based automation suites like `nvmetcli`, the agent creates Subsystems, Namespaces, and RDMA Ports strictly by writing to the Linux Kernel’s ConfigFS mount (`/sys/kernel/config/nvmet`). This guarantees atomic block-level exports directly from the kernel level.

Volume teardown employs Kubernetes *Finalizers*, intercepting NVMePartition deletion events to cleanly un-export the Fabric pathway before dropping the raw partition chunks to reclaim space.

4.3 Deployment Mechanics

To package the solution for end users, DISTORT maintains an integrated Helm Chart (`deploy/charts/distort`). The chart templates encapsulate:

- `crds/`: Pre-generated API models.
- `manager-deployment.yaml`: Deploying the centralized scheduler with leader election.
- `agent-daemonset.yaml`: A privileged DaemonSet mounting the host’s `/sys`, `/dev`, and `/sys/kernel/config` spaces.
- `csi-daemonset.yaml`: Encapsulating the `distort-csi` Node-Server alongside standard Kubernetes SIG-Storage sidecars (e.g., `csi-node-driver-registrar`).

4.4 Testing and Validation Methodology

To ensure the robustness of the storage engine across the entire stack—from hardware discovery to CSI volume provisioning—DISTORT employs a comprehensive end-to-end (E2E) testing framework. The environment is simulated using Vagrant and VirtualBox, provisioning a multi-node K3s cluster. Each virtual node is equipped with multiple virtual NVMe drives and configured with SoftRoCE (`rdma_rxe`) to emulate RDMA fabrics without requiring specialized hardware.

The validation suite is built upon the Ginkgo BDD (Behavior-Driven Development) testing framework and orchestrates tests across three functional layers:

- (1) **Hardware Discovery:** Asserts that the `distort-agent` accurately detects virtual NVMe disks and publishes corresponding `NVMeDevice` CRDs.
- (2) **Controller Logic:** Validates the `distort-manager`'s scheduling decisions, ensuring `NVMePartition` resources are placed on nodes with sufficient capacity.
- (3) **CSI Provisioning & Persistence:** Tests the full lifecycle: dynamic PVC creation, NVMe-oF target export via kernel `ConfigFS`, RDMA connection from the client pod, 'ext4' filesystem formatting, and data persistence guarantees upon pod restarts.

5 EVALUATION METHODOLOGY

While a comprehensive performance evaluation is left for future work, we outline a rigorous methodology to assess DISTORT's efficacy. The primary goal of DISTORT is to provide the operational flexibility of cloud-native storage while preserving the near bare-metal performance characteristics of RDMA-backed NVMe.

5.1 Performance Baselines

DISTORT should be evaluated against three distinct storage configurations to contextualize its performance profile:

- (1) **Local Bare-Metal NVMe:** Represents the theoretical maximum achievable throughput and the absolute minimum latency. This serves as the upper-bound baseline.
- (2) **Standard Network Block Device (NBD / iSCSI):** Provides a traditional, non-RDMA network storage baseline over TCP/IP, highlighting the latency benefits of the fabric interconnect.
- (3) **Commercial/SDS CSI Solutions (e.g., Ceph RBD, OpenEBS):** Represents popular software-defined storage deployments in Kubernetes, demonstrating the overhead introduced by user-space file systems and complex control planes compared to DISTORT's lightweight native configs approach.

5.2 Metrics and Workloads

The primary evaluation tool will be `fio` (Flexible I/O Tester), deployed as Kubernetes Pods consuming DISTORT-provisioned volumes. The targeted metrics include:

- **Throughput (IOPS & Bandwidth):** Measured using varying block sizes (e.g., 4KB for IOPS, 1MB for bandwidth) under highly concurrent random read/write workloads.

- **Latency Distributions:** Analyzing 99th and 99.9th percentile latencies critical for performance-sensitive applications like distributed databases.
- **Control Plane Overhead:** Measuring the end-to-end time required to dynamically provision and attach a volume (from PVC creation to Pod 'Running' state) to ensure the CRD-driven orchestration scales effectively under cluster churn.

6 CONCLUSION

This report presented DISTORT, a novel storage engine designed to seamlessly integrate high-performance, disaggregated NVMe-oF/RDMA storage with Kubernetes. By mapping physical hardware operations to declarative Custom Resources, DISTORT provides dynamic volume provisioning without sacrificing the bare-metal performance characteristics of RDMA, paving the way for data-intensive applications in cloud-native environments.